

An Empirical Evaluation of Application-Level Idempotency Strategies for Retry Correctness on AWS Lambda and Amazon DynamoDB

Vikas Reddy Amanagantti | Student ID: X25178849

MSc Cloud Computing — National College of Ireland — Short Explanation Guide

Baseline: Qi et al. Halfmoon (2025) ACM TOCS — DOI 10.1145/3725985

Read aloud in simple English. Numbers come from STATUS.md / results — do not invent live AWS metrics.

1. What is the project?

This project measures a practical fear: when AWS Lambda is retried, does your DynamoDB write happen twice? Managed Lambda often gives at-least-once execution.

Three paths on stock Lambda+DynamoDB: P1 plain Put (no guard); P2 conditional Put (attribute_not_exists); P3 idempotency-key (IDEMP#request_id then business write then COMPLETED). P4 TransactWrite is quarantined / out of scope.

Live campaign eu-west-1: $N=1000 \times \{P1,P2,P3\} \times \{1,2,5\} = 9000$ requests / 24000 deliveries; E1–E3 supported; stack destroyed.

Pitch: “I injected known retries into Lambda writing to DynamoDB three ways and measured duplicate mutations, latency, and capacity on stock AWS — I did not install Halfmoon.”

2. Problem statement + real-life example

If a function times out after the write committed, the platform may invoke again. Plain PutItem doubles the side effect — duplicate charges or orders. Runtime research solves this by changing the runtime; managed Lambda users cannot. They need application-level guards with measured costs.

Real-life example — bank transfer double-click. P1 = teller posts every click. P2 = post only if slip number is new. P3 = stamp request ID in a log book first, then post; second click finds the stamp. Halfmoon rebuilds the ledger machine; I only change how the teller writes.

3. Baseline paper

Qi et al. (2025), ACM TOCS (DOI 10.1145/3725985); SOSP'23 Halfmoon precursor (DOI 10.1145/3600006.3613154). Exactly-once via asymmetric logging on a custom serverless runtime; DynamoDB used inside that runtime with versioned conditionals.

Gap: Halfmoon is not installable on stock managed Lambda. We share the correctness criterion (no duplicate mutation after retry) and measure P1/P2/P3 on unmodified AWS. We do not reimplement Halfmoon or cite its overhead numbers as ours.

4. How we solve it / research question

RQ: By how much do conditional write (P2) and idempotency-key (P3) reduce duplicate state mutation versus plain write (P1) under injected Lambda timeouts — and what do they cost in latency and consumed capacity?

Driver records (request_id, intended_delivery_count); forces after-commit timeout; redelivers. DynamoDB Streams = ground truth. E1: P1 majority dups on retry. E2: P2/P3 cut dups >90% vs P1. E3: P3 costs more capacity than P2. All supported live.

Headline: P1 dup_rate=1.0 at mult 2 and 5; P2/P3=0.0; $P3 = P2 + 2 \text{ WCU/request}$. Sensitivity p3_between: P3 fails if crash between business write and COMPLETED. Must NOT claim: ran Halfmoon; P4 wins; moto latency is AWS.

5. Dataset / data

Fully synthetic payloads. No real users. Injected retries — not CloudWatch inference.

- Known (request_id, intended_delivery_count) before each invoke.
- Live: seed 25178849; N=1000/cell; mult {1,2,5}; workers 6; eu-west-1; on-demand; 0 driver errors.
- Streams GT: 21387 campaign records; CloudWatch invocations match.
- Sensitivity: 400 req / 1400 deliveries for p3_between.
- Moto factorial = plumbing only — never cite as AWS latency/capacity.

6. Implementation + technologies / AWS services

- AWS Lambda — one function, path switch P1|P2|P3.
- Amazon DynamoDB — business table + idempotency-key items.
- DynamoDB Streams — mutation ground truth.
- Terraform (infra/) — deploy/destroy (8 resources).
- Python driver — after-commit timeout injection.
- CloudWatch — invocation cross-check.
- pytest — 63 tests; moto plumbing only.
- Analysis — z-tests/ χ^2 /Mann–Whitney + Holm; live figures.

7. End-to-end flow

- Terraform apply → Lambda + DynamoDB + Streams + IAM.
- Driver picks path and multiplicity.
- First invoke writes; optional after-commit timeout.
- Redeliver same request_id as planned.
- P1 may mutate again; P2 fails condition; P3 skips on key.
- Streams + logs → dup rate, latency, WCU/RCU.
- Full factorial N=1000; analyse E1–E3; destroy stack.

8. Exact speaking script (dataset + implementation + technologies)

“My dataset is synthetic Lambda requests with known retry counts that I inject myself — not real users, and not guessed from CloudWatch. I implemented one Lambda that can write to DynamoDB three ways: plain put, conditional put, and an idempotency-key pattern. DynamoDB Streams give me ground truth of every mutation.”

“Infrastructure is Terraform. On the live campaign I ran one thousand requests for each path and each retry multiplicity in eu-west-1, then destroyed the stack. Technologies are Python, AWS Lambda, DynamoDB, Streams, and standard statistics. I did not install Halfmoon; I only share its correctness idea on stock AWS.”

9. STATUS snapshot + five Q&A

Campaign	N=1000×3×3 = 24000 deliveries; 0 driver errors
Dup rates	P1=1.0 at mult 2/5; P2/P3=0.0
E1-E3	all supported on live
Destroy	8 TF resources destroyed

Q: Did you run Halfmoon?

A: No — baseline idea only; stock managed Lambda+DynamoDB.

Q: What is P4?

A: TransactWrite — quarantined / out of scope for CA2.

Q: Cite moto as AWS?

A: No — moto is plumbing; live campaign answers the RQ.

Q: Did guarded paths stop duplicates?

A: Yes: P2/P3 dup_rate=0.0; P1=1.0 at mult 2 and 5.

Q: When does P3 still fail?

A: p3_between: crash between business write and COMPLETED mark.